

Regresión simbólica para la expresión analítica de soluciones de ecuaciones no lineales

Noel Pérez Calvo

28 de abril de 2026

Índice

1. Preliminares	4
1.1. Sistemas de ecuaciones no lineales paramétricos	4
1.1.1. Definición y notación	4
1.1.2. Métodos numéricos para el cálculo de ceros	5
1.1.3. Distancia euclidiana	6
1.1.4. Producto cartesiano	7
1.2. Regresión simbólica	8
1.2.1. Concepto y objetivos	8
1.2.2. Fundamentos de programación genética	9
1.2.3. Funciones de pérdida	10
1.2.4. Parámetros del algoritmo de regresión simbólica	11
2. Propuesta de solución	13
2.1. Planteamiento general del problema	13
2.2. Generación de datos de entrenamiento	14
2.2.1. Muestreo del espacio de parámetros	14
2.2.2. Resolución numérica del sistema	14
2.2.3. Filtrado de soluciones duplicadas	15
2.3. Estrategia de regresión simbólica	15
2.3.1. Separación por coordenadas	16
2.3.2. Búsqueda iterativa sobre x_1	16
2.3.3. Extensión a las demás coordenadas	17
2.3.4. Validación de una solución completa	18
2.3.5. Funciones de pérdida utilizadas	19
2.4. Pseudocódigo del proceso principal	20
2.5. Implementación del sistema	21

2.5.1.	Descripción de los módulos	21
2.5.2.	Entradas, parámetros y salidas del sistema	23
3.	Experimentación y resultados	24
4.	Conclusiones	25

Introducción

1. Preliminares

Este capítulo establece los fundamentos teóricos necesarios para abordar el problema de descubrir expresiones analíticas de los ceros de sistemas de ecuaciones no lineales mediante regresión simbólica. Para ello se presentan los fundamentos conceptuales, las herramientas numéricas y las estrategias algorítmicas que sustentan la metodología propuesta en este trabajo.

En la primera parte del capítulo (1.1) se introduce la notación y las propiedades de los sistemas de ecuaciones no lineales paramétricos, destacando la multiplicidad de soluciones que pueden existir para una misma tupla de parámetros. Se describen los métodos numéricos para el cálculo de ceros de tales sistemas, tanto los métodos iterativos clásicos como el método híbrido de Powell implementado en la biblioteca `SciPy` [9]. Se definen formalmente los conceptos de distancia euclidiana y producto cartesiano, que serán utilizados en la generación del conjunto de datos de entrenamiento.

Posteriormente, en la sección 1.2 se abordan los principios de la regresión simbólica [6], una técnica de aprendizaje automático que permite inferir expresiones algebraicas a partir de datos. Se explican los fundamentos de la programación genética, las funciones de pérdida que guían el proceso evolutivo y los parámetros que controlan el comportamiento del algoritmo.

1.1. Sistemas de ecuaciones no lineales paramétricos

En esta sección se formaliza el concepto de sistema de ecuaciones no lineales dependiente de parámetros, así como las herramientas numéricas y matemáticas necesarias para su estudio.

1.1.1. Definición y notación

Un sistema de ecuaciones no lineales paramétrico se expresa como la búsqueda de vectores $\mathbf{x} \in \mathbb{R}^n$ que satisfacen

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}, \quad (1)$$

donde:

- $\mathbf{F} : \mathbb{R}^n \times \mathbb{R}^p \rightarrow \mathbb{R}^n$ es una función vectorial no lineal,
- $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ es el vector de variables incógnita,
- $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)^T \in \mathbb{R}^p$ es el vector de parámetros del modelo,
- $\mathbf{0} \in \mathbb{R}^n$ denota el vector nulo.

La notación $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})$ indica que $\mathbf{F}$ depende explícitamente de las variables $\mathbf{x}$ y de los parámetros $\boldsymbol{\theta}$, estos últimos considerados fijos durante el proceso de resolución.

Ejemplo 1. Considérese el sistema de dos ecuaciones

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \begin{pmatrix} (x_1 - a)(x_2 - ab) \\ x_1x_2 - ab^2 \end{pmatrix} = \mathbf{0},$$

con $\mathbf{x} = (x_1, x_2)^T \in \mathbb{R}^2$ y parámetros $\boldsymbol{\theta} = (a, b)^T$, $a > 0$, $b > 0$. Este sistema posee dos soluciones reales para cada tupla de parámetros: $\mathbf{x} = (a, b^2)$ y $\mathbf{x} = (b, ab)$. Nótese que la primera componente de la solución (a, b^2) anula el factor $(x_1 - a)$ de la primera ecuación, mientras que en la solución (b, ab) es la segunda componente la que anula el factor $(x_2 - ab)$. Esta multiplicidad de ceros y su dependencia explícita de los parámetros ilustran el tipo de fenómenos que aparecen en sistemas no lineales paramétricos y motivan la necesidad de determinar, para cada valor de los parámetros, todas las raíces que serán utilizadas en la construcción del conjunto de datos de entrenamiento.

La no linealidad de $\mathbf{F}$ implica que, a diferencia de los sistemas lineales, no existe un método directo general para obtener todas las soluciones, y la estructura del conjunto solución puede ser compleja. Por esta razón se recurre a métodos numéricos iterativos, que se describen en la siguiente sección.

1.1.2. Métodos numéricos para el cálculo de ceros

Una vez definido el sistema no lineal paramétrico, es necesario obtener numéricamente sus soluciones para distintas tuplas de parámetros. Dado que, salvo en casos triviales, no es posible despejar analíticamente las raíces de un sistema no lineal arbitrario, se recurre a métodos iterativos que aproximan las soluciones de manera sucesiva. A continuación se describen dos de los métodos más utilizados, que serán empleados en la etapa de generación de datos.

Para sistemas no lineales generales, la obtención de soluciones explícitas mediante métodos simbólicos no es factible, ya que la mayoría de estos sistemas carecen de soluciones expresables en forma cerrada [3]. En su lugar, se emplean métodos numéricos iterativos que, partiendo de una aproximación inicial $\mathbf{x}^{(0)}$, generan una sucesión $\{\mathbf{x}^{(k)}\}_{k=0}^{\infty}$ que converge a una solución $\mathbf{x}^*$, siempre que la aproximación inicial esté suficientemente cerca de dicha solución y que $\mathbf{F}$ sea diferenciable con jacobiano no singular en $\mathbf{x}^*$ [7].

El método de Newton-Raphson para sistemas no lineales [3] se define mediante la iteración

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - [\mathbf{J}_{\mathbf{F}}(\mathbf{x}^{(k)})]^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta}), \quad (2)$$

donde:

- $\mathbf{x}^{(k)}$ es la aproximación en la iteración k ,
- $\mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta})$ es el valor de $\mathbf{F}$ evaluado en dicha aproximación,
- $\mathbf{J}_{\mathbf{F}}(\mathbf{x})$ es la matriz jacobiana de $\mathbf{F}$ respecto de $\mathbf{x}$, de dimensión $n \times n$, con entradas $(\mathbf{J}_{\mathbf{F}})_{ij} = \frac{\partial F_i}{\partial x_j}$.

Si $\mathbf{F}$ es diferenciable y $\mathbf{J}_{\mathbf{F}}$ es no singular en una solución $\mathbf{x}^*$, entonces existe un entorno de $\mathbf{x}^*$ donde el método converge cuadráticamente, es decir, $\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \leq C\|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2$ para alguna constante $C > 0$ [7].

Los métodos cuasi-Newton [7] reducen el costo computacional al evitar el cálculo exacto del jacobiano en cada iteración. En lugar de ello, construyen una aproximación $\mathbf{B}_k$ de la matriz jacobiana que se actualiza con información de los gradientes evaluados en iteraciones previas. Las actualizaciones típicas son de rango uno o dos, lo que significa que modifican la matriz anterior sumando una matriz de rango uno o dos, respectivamente. La iteración toma la forma

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{B}_k^{-1} \mathbf{F}(\mathbf{x}^{(k)}; \boldsymbol{\theta}),$$

donde $\mathbf{B}_k$ se actualiza mediante fórmulas como la de Broyden [2]. Estos métodos alcanzan una convergencia superlineal: la tasa de reducción del error es más rápida que la lineal pero sin llegar a la velocidad cuadrática del método de Newton completo.

En este trabajo se utilizará un método híbrido, como el implementado en la biblioteca SciPy [9], que combina una estrategia cuasi-Newton con búsqueda de paso para mejorar la robustez. A partir de este procedimiento se generará el conjunto de datos de entrenamiento para la regresión simbólica, el cual está formado por múltiples tuplas $(\boldsymbol{\theta}, \mathbf{x}^*)$, donde $\mathbf{x}^*$ es una raíz obtenida numéricamente para el parámetro $\boldsymbol{\theta}$.

1.1.3. Distancia euclidiana

En la etapa de resolución numérica, dos raíces obtenidas a partir de puntos iniciales distintos pueden corresponder al mismo cero del sistema, diferenciándose únicamente por errores de redondeo o por las tolerancias de convergencia de los métodos iterativos. Para decidir si dos soluciones deben considerarse iguales, se utiliza la distancia euclidiana.

Definición 1 (Distancia euclidiana). Sean $\mathbf{u} = (u_1, \dots, u_n)^T \in \mathbb{R}^n$ y $\mathbf{v} = (v_1, \dots, v_n)^T \in \mathbb{R}^n$ dos vectores. La **distancia euclidiana** entre $\mathbf{u}$ y $\mathbf{v}$ se

define como la norma euclidiana de su diferencia:

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^n (u_i - v_i)^2}. \quad (3)$$

La distancia euclidiana constituye una métrica sobre $\mathbb{R}^n$. En este trabajo, dos soluciones $\mathbf{x}$ y $\mathbf{x}'$ se consideran numéricamente equivalentes si su distancia euclidiana es menor que una tolerancia prefijada $\tau > 0$, lo que típicamente ocurre cuando ambas corresponden al mismo cero del sistema pero difieren debido a la acumulación de errores numéricos.

En particular, la **norma euclidiana** de un vector $\mathbf{v} \in \mathbb{R}^n$ se define como su distancia al vector nulo, es decir,

$$\|\mathbf{v}\|_2 = d(\mathbf{v}, \mathbf{0}) = \sqrt{\sum_{i=1}^n v_i^2}. \quad (4)$$

Esta cantidad resulta útil para medir qué tan cerca está un punto de ser solución del sistema: si $\|\mathbf{F}(\mathbf{x}; \boldsymbol{\theta})\|_2$ es pequeño, entonces $\mathbf{x}$ está próximo a anular todas las ecuaciones.

Una vez identificadas las soluciones distintas para cada tupla de parámetros, es necesario organizar sistemáticamente los valores de dichos parámetros sobre los que se evaluará el sistema. Esta organización se realiza mediante el producto cartesiano, que se define a continuación.

1.1.4. Producto cartesiano

Para construir el conjunto de datos de entrenamiento se debe evaluar el sistema no lineal en una variedad representativa de valores de los parámetros. Con este fin se selecciona, para cada parámetro θ_i , un conjunto finito de valores dentro de su rango admisible; el producto cartesiano de estos conjuntos proporciona todas las combinaciones posibles de parámetros que se utilizarán en la etapa de resolución numérica.

Definición 2 (Producto cartesiano). *Sean $A_1, A_2, \dots, A_m$ conjuntos arbitrarios. El **producto cartesiano** de estos conjuntos se define como el conjunto de todas las m -tuplas ordenadas $(a_1, a_2, \dots, a_m)$ tales que $a_i \in A_i$ para cada $i = 1, \dots, m$. Formalmente:*

$$A_1 \times A_2 \times \dots \times A_m = \{(a_1, a_2, \dots, a_m) \mid a_i \in A_i, i = 1, \dots, m\}. \quad (5)$$

Si cada A_i es un conjunto finito, el cardinal del producto cartesiano es el producto de los cardinales: $|A_1 \times \cdots \times A_m| = \prod_{i=1}^m |A_i|$. En el contexto de este trabajo, cada A_i es un conjunto de valores del parámetro θ_i , típicamente elegidos de manera equiespaciada dentro de un intervalo $[\theta_i^{\min}, \theta_i^{\max}]$. El producto cartesiano de estos conjuntos genera todas las tuplas $\boldsymbol{\theta} = (\theta_1, \dots, \theta_p)$ que serán utilizadas como entrada del sistema durante la generación de datos, garantizando una cobertura uniforme de la región de interés en el espacio de parámetros.

Una vez definidas las herramientas para generar los valores de los parámetros y para identificar las raíces del sistema, se dispone del marco necesario para producir el conjunto de datos con el que se alimentará el algoritmo de regresión simbólica, cuyos fundamentos se exponen en la siguiente sección.

1.2. Regresión simbólica

La regresión simbólica es la técnica central sobre la que se apoya la metodología de este trabajo. A continuación se presentan sus fundamentos teóricos y los elementos que la componen.

1.2.1. Concepto y objetivos

La regresión simbólica es una técnica de aprendizaje automático que descubre, a partir de un conjunto de datos observados, una expresión matemática cerrada. Dicha expresión describe la relación funcional entre las variables de entrada y la variable de salida [6]. A diferencia de la regresión tradicional, donde la estructura del modelo se fija de antemano y solo se ajustan los parámetros, la regresión simbólica explora simultáneamente el espacio de formas funcionales y de parámetros numéricos [8].

Existen diversas implementaciones de regresión simbólica, muchas de las cuales se basan en programación genética, enfoque que se describe en la siguiente subsección. Una de las más utilizadas es **PySR** [4], una biblioteca de código abierto que combina un motor evolutivo en Julia con una interfaz en Python, ofreciendo operadores matemáticos protegidos —versiones de funciones como la raíz cuadrada o la división que evitan valores indefinidos (NaN o infinitos) durante la búsqueda— para garantizar la estabilidad numérica.

Dado un conjunto de datos $\mathcal{D} = \{(\mathbf{u}_i, v_i)\}_{i=1}^N$ donde:

- $\mathbf{u}_i \in \mathbb{R}^d$ es el vector de variables de entrada del i -ésimo punto del conjunto de datos,
- $v_i \in \mathbb{R}$ es el valor observado de la variable de salida para dicho punto,

se busca una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$ dentro de un espacio de funciones $\mathcal{F}$ que minimice una función de pérdida $\mathcal{L}$, la cual mide la discrepancia entre las predicciones del modelo y los valores observados:

$$f^* = \arg \min_{f \in \mathcal{F}} \mathcal{L}(f; \mathcal{D}). \quad (6)$$

El espacio $\mathcal{F}$ está compuesto por todas las funciones que pueden construirse combinando un conjunto de operadores matemáticos básicos (suma, resta, multiplicación, división, logaritmos, etc.) y constantes numéricas. La tarea de encontrar la función óptima es intrínsecamente combinatoria, pues el número de expresiones que pueden formarse crece de manera exponencial con la cantidad de operadores y variables disponibles. Por esta razón se requieren métodos de búsqueda heurística, como los algoritmos evolutivos.

La implementación más extendida de estos algoritmos para regresión simbólica es la programación genética, cuyos fundamentos se exponen a continuación.

1.2.2. Fundamentos de programación genética

Una de las estrategias más utilizadas para la regresión simbólica es la **programación genética**, una rama de los algoritmos evolutivos inspirada en la selección natural [6]. Existen también enfoques basados en recocido simulado, gramáticas estocásticas y redes neuronales, aunque la programación genética sigue siendo una de las opciones más extendidas por su flexibilidad para explorar espacios de expresiones algebraicas.

En este paradigma, cada posible expresión matemática se codifica como un **árbol sintáctico**: una estructura jerárquica en la que los nodos internos contienen operadores matemáticos (suma, multiplicación, seno, etc.) y las hojas contienen variables de entrada o constantes numéricas. La figura resultante tiene forma de árbol invertido, donde la raíz es el operador principal y las ramas conducen a los operandos.

La programación genética trabaja con una población de individuos (árboles) que se modifica a lo largo de generaciones —ciclos iterativos del proceso evolutivo— mediante los siguientes operadores:

- **Selección:** Los individuos que producen predicciones más cercanas a los valores reales tienen mayor probabilidad de ser elegidos como padres para la siguiente generación.
- **Cruce:** Dos árboles padres intercambian algunos de sus subárboles para generar dos hijos. Existen diversas variantes; una de las más comunes es el cruce por subárbol, en el que se elige aleatoriamente un nodo de cada padre y se intercambian los subárboles correspondientes.

- **Mutación:** Se introduce una modificación aleatoria en un individuo para mantener la diversidad genética. Los tipos de mutación incluyen la sustitución de un subárbol por otro generado al azar, la alteración de un único nodo (cambiando un operador por otro), o la modificación de una hoja (cambiando una variable o constante).
- **Elitismo:** Una fracción de los mejores individuos de cada generación se preserva sin cambios en la siguiente, evitando así que se pierdan las soluciones más prometedoras. Este mecanismo es opcional y su intensidad depende del problema.

La repetición de estos operadores a lo largo de las generaciones permite explorar el espacio de posibles expresiones algebraicas, guiando la búsqueda hacia funciones que se ajustan cada vez mejor a los datos [6].

La calidad de cada individuo se mide mediante una función de pérdida, cuyas variantes más relevantes para este trabajo se presentan a continuación.

1.2.3. Funciones de pérdida

La **función de pérdida** —también llamada función de aptitud en el contexto de los algoritmos evolutivos— es el criterio que se utiliza para evaluar cuantitativamente qué tan buena es una expresión candidata producida por la regresión simbólica. Esta función mide la discrepancia entre las predicciones de la expresión simbólica y los valores observados en el conjunto de datos. Una función de pérdida común, y la que se emplea como referencia en este trabajo, es el error cuadrático medio.

Error cuadrático medio. Para un conjunto de datos $\{(\mathbf{u}_i, v_i)\}_{i=1}^N$ con $\mathbf{u}_i \in \mathbb{R}^d$ y $v_i \in \mathbb{R}$, y una función $f : \mathbb{R}^d \rightarrow \mathbb{R}$, el error cuadrático medio (MSE, por sus siglas en inglés) se define como

$$\text{MSE}(f) = \frac{1}{N} \sum_{i=1}^N (v_i - f(\mathbf{u}_i))^2, \quad (7)$$

donde:

- N es el número total de puntos en el conjunto de datos,
- v_i es el valor observado para el i -ésimo punto,
- $f(\mathbf{u}_i)$ es el valor predicho por la expresión f para el vector de entrada $\mathbf{u}_i$.

El MSE penaliza cuadráticamente las desviaciones, por lo que es sensible a valores atípicos [5]. En el contexto de la regresión simbólica, es una de las funciones de pérdida más utilizadas debido a su simplicidad y a que proporciona una señal suave para guiar el proceso de búsqueda [1].

La efectividad del proceso evolutivo no solo depende de la función de pérdida elegida, sino también de la configuración del algoritmo. A continuación se describen los parámetros que controlan el comportamiento de la programación genética aplicada a la regresión simbólica.

1.2.4. Parámetros del algoritmo de regresión simbólica

El comportamiento de la regresión simbólica basada en programación genética depende de un conjunto de parámetros que influyen en la calidad de las soluciones, el tiempo de cómputo y la interpretabilidad de los resultados [4]. A continuación se describen los más relevantes, agrupados según el aspecto del algoritmo que controlan.

Parámetros de la población.

- **Tamaño de la población (P):** Número de individuos que coexisten en cada generación. Una población grande aumenta la diversidad genética y la probabilidad de encontrar soluciones de alta calidad, a cambio de un mayor tiempo de cómputo y consumo de memoria.
- **Número de generaciones (G):** Cantidad de iteraciones del proceso evolutivo. Valores más altos permiten refinar las soluciones, pero pueden conducir a sobreajuste si no se limita la complejidad de las expresiones descubiertas.
- **Número de poblaciones independientes:** En algunas implementaciones se utiliza un modelo de islas, que consiste en dividir la población total en varias subpoblaciones que evolucionan en paralelo. Periódicamente, un pequeño número de individuos migra entre las islas, lo que ayuda a mantener la diversidad genética y evita la convergencia prematura.

Parámetros de control de la complejidad. Además de la precisión, interesa obtener expresiones sencillas que sean fáciles de interpretar. Los siguientes parámetros regulan el equilibrio entre ajuste a los datos y simplicidad de las fórmulas.

- **Coeficiente de parsimonia (α):** Penalización añadida a la función de aptitud, proporcional al tamaño del árbol (número de nodos). La

aptitud efectiva —es decir, el valor que el algoritmo optimiza— se calcula como $\text{fitness} = \text{MSE} + \alpha \cdot \text{tamaño}$. Un valor mayor de α favorece expresiones más simples.

- **Tamaño máximo del árbol** ($S_{\text{máx}}$): Límite en el número de nodos que puede tener un individuo. Evita la explosión de la complejidad y reduce el espacio de búsqueda.

Probabilidades de operadores genéticos. La composición de cada nueva generación está determinada por las siguientes probabilidades, que deciden qué operador se aplica a cada individuo.

- **Probabilidad de cruce** (p_c): Fracción de la nueva generación generada mediante cruce de padres seleccionados.
- **Probabilidad de mutación** (p_m): Fracción de individuos que sufren modificaciones aleatorias. Existen distintos tipos de mutación (de subárbol, de punto, de hoja) cuyas probabilidades individuales pueden ajustarse.
- **Probabilidad de reproducción** (p_r): Fracción de individuos que pasan directamente a la siguiente generación sin cambios (elitismo), con $p_r = 1 - p_c - p_m$.

Con esta base, en el siguiente capítulo se desarrollará la metodología propuesta, que integra los conceptos aquí descritos para construir y entrenar modelos de regresión simbólica capaces de recuperar las expresiones analíticas de las raíces a partir de datos numéricos.

2. Propuesta de solución

En este capítulo se desarrolla la metodología propuesta para recuperar las expresiones analíticas de los ceros de sistemas de ecuaciones no lineales paramétricos. A diferencia de una resolución puramente numérica, que proporciona valores puntuales para cada tupla de parámetros, se busca descubrir las funciones que relacionan dichos parámetros con las raíces del sistema. Este objetivo plantea dos desafíos principales: la obtención de un conjunto de datos suficientemente representativo del comportamiento del sistema y la inferencia de las formas funcionales a partir de esos datos.

Para abordar el primer desafío, se describe en la Sección 2.2 la construcción del conjunto de entrenamiento a partir de un muestreo del espacio de parámetros y la resolución numérica de las ecuaciones, incluyendo un paso de filtrado para eliminar soluciones redundantes. El segundo desafío se enfrenta en la Sección 2.3, donde se expone la estrategia de regresión simbólica multidimensional. Esta estrategia, que constituye el núcleo del trabajo, emplea una búsqueda iterativa que descubre progresivamente las distintas ramas del conjunto solución, validando cada expresión frente a los datos restantes y utilizando una función de pérdida especialmente adaptada al problema. El capítulo concluye con el pseudocódigo completo del algoritmo (Sección 2.4) y una descripción de los módulos que componen la implementación (Sección 2.5).

2.1. Planteamiento general del problema

El objetivo central de este trabajo es transformar la resolución numérica del sistema

$$\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}, \quad (8)$$

definido en la Sección 1.1, en la tarea de encontrar todas las expresiones analíticas que describen sus soluciones.

Para un valor fijo de los parámetros $\boldsymbol{\theta}$, el sistema puede tener más de una solución. En lugar de calcular numéricamente esas soluciones para cada $\boldsymbol{\theta}$ por separado, lo que se busca es obtener las funciones que, dado $\boldsymbol{\theta}$, devuelven directamente cada una de las soluciones posibles. Como estas soluciones forman naturalmente grupos que varían de manera continua con los parámetros, las organizamos en una lista indexada:

$$\mathbf{x} = \mathbf{g}^{(1)}(\boldsymbol{\theta}), \quad \mathbf{x} = \mathbf{g}^{(2)}(\boldsymbol{\theta}), \quad \dots$$

Aquí, $\mathbf{g}^{(k)}$ es una función que toma parámetros y devuelve un vector de incógnitas; el superíndice k simplemente distingue las distintas soluciones

que pueden coexistir para una misma θ . Con este planteamiento, el problema queda reducido a descubrir automáticamente, a partir de datos, las expresiones matemáticas de esas funciones $\mathbf{g}^{(k)}$.

Mientras que los métodos numéricos tradicionales proporcionan soluciones puntuales para valores fijos de los parámetros [3], el enfoque propuesto persigue capturar la dependencia funcional completa. Esto permite evaluar las soluciones para nuevos parámetros sin necesidad de repetir el proceso numérico, y facilita el análisis cualitativo del sistema al mostrar explícitamente cómo varían las raíces.

2.2. Generación de datos de entrenamiento

La primera etapa del método consiste en construir un conjunto de datos que relacione los parámetros del sistema con sus soluciones. Este conjunto se obtiene muestreando el espacio de parámetros, resolviendo el sistema para cada tupla y filtrando las soluciones duplicadas.

2.2.1. Muestreo del espacio de parámetros

Para cada parámetro θ_i se elige un conjunto finito Θ_i de valores equiespaciados dentro de un intervalo $[\theta_i^{\min}, \theta_i^{\max}]$. El producto cartesiano

$$\mathcal{G} = \Theta_1 \times \Theta_2 \times \cdots \times \Theta_p$$

genera todas las combinaciones posibles de parámetros. Esta construcción se apoya en la definición de producto cartesiano presentada en la sección 1.1 y produce una cuadrícula regular que cubre uniformemente la región de interés.

2.2.2. Resolución numérica del sistema

Para cada tupla $\theta \in \mathcal{G}$ se resuelve el sistema $\mathbf{F}(\mathbf{x}; \theta) = \mathbf{0}$ mediante la función `root` de la biblioteca `SciPy` [9]. Esta función se invoca con el método predeterminado, que corresponde al esquema híbrido discutido en la sección 1.1. El solucionador devuelve un objeto que contiene un campo booleano `success` (que indica si la iteración convergió) y el vector solución `s.x`.

Dado que pueden existir varias soluciones reales para un mismo valor de los parámetros, se ejecuta el método numérico desde múltiples puntos iniciales aleatorios. Cada punto inicial se genera eligiendo cada coordenada x_i aleatoriamente dentro de un intervalo predefinido $[x_i^{\min}, x_i^{\max}]$. Se realiza un número fijo de intentos. Si el método converge y se cumple que

$$\|\mathbf{F}(\mathbf{s.x}; \theta)\|_2 < \varepsilon_{\text{res}},$$

donde ε_{res} es una tolerancia que indica qué tan cerca debe estar $\mathbf{F}$ del vector nulo para considerar que el punto encontrado es una solución válida, se acepta el punto y se agrega a la lista de soluciones encontradas para ese valor de los parámetros. Los valores concretos del número de intentos, los intervalos y las tolerancias se establecerán durante la experimentación.

2.2.3. Filtrado de soluciones duplicadas

Los puntos iniciales aleatorios pueden producir la misma solución real, sobre todo si se parte de regiones cercanas. Para evitar duplicados, se aplica un filtro basado en la distancia euclidiana (sección 1.1): dos vectores de solución $\mathbf{u}$ y $\mathbf{v}$ se consideran idénticos si

$$\|\mathbf{u} - \mathbf{v}\|_2 < \tau,$$

donde τ es una tolerancia prefijada que depende de la escala del problema y de la precisión de la máquina.

Tras este proceso se dispone de un conjunto de pares $(\boldsymbol{\theta}, \mathbf{x}^*)$, donde $\mathbf{x}^*$ es una solución real y única del sistema para los parámetros $\boldsymbol{\theta}$. Este conjunto constituye el dato de entrada para la etapa de regresión simbólica.

2.3. Estrategia de regresión simbólica

Una vez construido el conjunto de datos que empareja cada tupla de parámetros con las soluciones numéricas del sistema, el paso siguiente es extraer, a partir de esos datos, las relaciones funcionales que los describen. Para ello se emplea regresión simbólica, que permite descubrir expresiones algebraicas cerradas en lugar de limitarse a interpolar valores.

La dificultad principal es que los datos contienen múltiples soluciones para una misma tupla de parámetros y que cada solución está formada por varias variables interdependientes. Abordar directamente el problema como un ajuste vectorial no es viable, porque el algoritmo tendería a promediar soluciones distintas. En su lugar, se sigue una estrategia en tres fases. Primero, se descompone el problema tratando cada coordenada de la solución como una función independiente de los parámetros y se elige una de ellas para guiar la separación de las distintas soluciones (sección 2.3.1). A continuación, mediante un proceso iterativo de regresión simbólica, se descubren una a una las expresiones para la coordenada elegida y, a partir de ellas, se extiende el ajuste al resto de las coordenadas (secciones 2.3.2 y 2.3.3). Finalmente, cada solución completa se valida comprobando que satisface el sistema original dentro de una tolerancia preestablecida (sección 2.3.4).

2.3.1. Separación por coordenadas

El conjunto de datos generado en la etapa anterior asocia cada tupla de parámetros θ con una o varias soluciones del sistema. Esta característica impide abordar el problema como un simple ajuste funcional: si para un mismo θ existen varios valores de $\mathbf{x}$, cualquier función que intente predecir $\mathbf{x}$ a partir de θ estará mal definida, pues a una misma entrada le corresponderían varias salidas. Un modelo de regresión entrenado directamente sobre estos datos tendería a producir un valor intermedio entre las distintas soluciones.

Para superar esta dificultad se descompone el problema: en lugar de buscar una función vectorial que asigne directamente θ al vector completo $\mathbf{x}$, se trata cada coordenada por separado. Así, el objetivo pasa a ser encontrar n funciones escalares

$$x_k = g_k(\theta), \quad k = 1, \dots, n,$$

todas ellas definidas sobre el mismo espacio de parámetros. La notación g_k representa la función que describe cómo la k -ésima coordenada de la solución depende de los parámetros. Con esta separación, el problema se reduce a hallar expresiones algebraicas para cada variable de forma independiente, aprovechando que la relación entre parámetros y cada coordenada de la solución sí puede ser funcional.

Sin embargo, los datos de una misma coordenada también contienen, en general, múltiples valores para un mismo θ , ya que provienen de soluciones distintas. Para poder aplicar la regresión simbólica es necesario primero separar estas soluciones. La estrategia que se adopta consiste en elegir una de las coordenadas como guía para agrupar los puntos. Sin pérdida de generalidad, se toma x_1 como punto de partida. La idea es que, una vez que se identifiquen las distintas funciones que describen a x_1 y se asigne cada punto del conjunto de datos a una de ellas, las demás coordenadas quedarán automáticamente agrupadas: para un mismo θ , el valor de cada x_k está asociado a la misma solución que el valor de x_1 .

De esta manera, el proceso se organiza en dos etapas: primero se aplica regresión simbólica sobre los datos de x_1 para descubrir, mediante un procedimiento iterativo, cuántas funciones distintas la describen y a qué subconjunto de puntos corresponde cada una; a continuación, se extiende el ajuste a las coordenadas restantes. Los detalles de cada etapa se exponen en las secciones siguientes.

2.3.2. Búsqueda iterativa sobre x_1

Una vez organizados los datos, se aplica el algoritmo de regresión simbólica iterativa descrito en la sección 1.2 al conjunto de pares (θ, x_1) . El objetivo

de esta etapa es descubrir, una a una, las distintas funciones que describen la primera coordenada de las soluciones.

El algoritmo comienza con el conjunto completo de puntos y, en cada iteración, entrena un modelo de regresión simbólica que busca una expresión capaz de ajustar la mayor cantidad posible de puntos dentro de una tolerancia preestablecida. La función obtenida en la j -ésima iteración se denota como $g_1^{(j)}(\boldsymbol{\theta})$, donde el superíndice (j) indica que se trata de la j -ésima solución descubierta para x_1 .

Los puntos que resultan correctamente explicados por $g_1^{(j)}$ se retiran del conjunto de datos, y la iteración siguiente trabaja exclusivamente con los puntos restantes. Este proceso se repite hasta que el número de puntos sin explicar es menor que un umbral mínimo, momento en el cual se considera que todas las soluciones relevantes han sido identificadas.

Cada expresión $g_1^{(j)}$ queda asociada de manera natural al subconjunto de parámetros que la satisfacen. Formalmente, se define

$$\mathcal{S}^{(j)} = \{ \boldsymbol{\theta} \in \mathcal{G} \mid g_1^{(j)}(\boldsymbol{\theta}) \text{ explica el punto } (\boldsymbol{\theta}, x_1) \},$$

es decir, el conjunto de tuplas de parámetros para las cuales la expresión $g_1^{(j)}$ se ajusta al valor observado de x_1 dentro de la tolerancia establecida.

Puesto que los valores de x_1 para una misma tupla de parámetros provienen de soluciones distintas, cada una de estas funciones identifica una solución del sistema. Al finalizar esta etapa se cuenta con una lista de funciones para x_1 , cada una con su subconjunto de parámetros asociado, lo que proporciona la base para extender el ajuste a las coordenadas restantes.

2.3.3. Extensión a las demás coordenadas

Una vez que la búsqueda iterativa ha proporcionado una lista de expresiones $g_1^{(1)}, g_1^{(2)}, \dots$ para la primera coordenada, junto con los subconjuntos de parámetros $\mathcal{S}^{(1)}, \mathcal{S}^{(2)}, \dots$ que cada una explica, el paso siguiente es completar la descripción de cada solución del sistema. Para ello se obtienen expresiones simbólicas para las coordenadas restantes $x_2, x_3, \dots, x_n$.

Para una solución particular, identificada por el índice j y caracterizada por la función $g_1^{(j)}$ y su subconjunto asociado $\mathcal{S}^{(j)}$, se dispone de un conjunto de puntos donde ya se conoce la relación entre los parámetros $\boldsymbol{\theta}$ y el valor de cada x_k , ya que los datos originales contienen el vector completo $\mathbf{x}$. Aprovechando esta información, se entrena un modelo de regresión simbólica simple (una sola ecuación) para cada coordenada $k = 2, \dots, n$, usando exclusivamente los puntos del subconjunto $\mathcal{S}^{(j)}$. La función obtenida para la coordenada k de la solución j se denota como $g_k^{(j)}(\boldsymbol{\theta})$.

Este procedimiento es directo porque, dentro de un mismo subconjunto, no existe la ambigüedad de múltiples soluciones para una misma tupla de parámetros: todos los puntos de $\mathcal{S}^{(j)}$ corresponden a la misma solución del sistema. Por tanto, la regresión simbólica puede aplicarse sin necesidad de estrategias iterativas adicionales.

Una vez completado este proceso para todos los subconjuntos, se dispone de un conjunto de funciones que describen cada solución del sistema. Para la solución j , las expresiones son

$$\begin{aligned} x_1 &= g_1^{(j)}(\boldsymbol{\theta}), \\ x_2 &= g_2^{(j)}(\boldsymbol{\theta}), \\ &\vdots \\ x_n &= g_n^{(j)}(\boldsymbol{\theta}). \end{aligned}$$

Estas funciones constituyen un candidato a descripción analítica de la solución, que debe ser validado antes de ser aceptado definitivamente.

2.3.4. Validación de una solución completa

Una vez obtenidas las expresiones simbólicas para todas las coordenadas de una candidata a solución, es necesario verificar que el conjunto de funciones

$$\mathbf{g}^{(j)}(\boldsymbol{\theta}) = (g_1^{(j)}(\boldsymbol{\theta}), g_2^{(j)}(\boldsymbol{\theta}), \dots, g_n^{(j)}(\boldsymbol{\theta}))$$

realmente satisface el sistema original $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}$ sobre el subconjunto de parámetros $\mathcal{S}^{(j)}$ que la generó. Esta comprobación es imprescindible, ya que las expresiones se han obtenido por ajuste independiente de cada coordenada y podrían no ser compatibles entre sí al ser evaluadas conjuntamente en el sistema.

Para realizar la validación se sustituyen las funciones simbólicas en el sistema y se evalúa el valor

$$\mathbf{F}(\mathbf{g}^{(j)}(\boldsymbol{\theta}); \boldsymbol{\theta})$$

para cada tupla $\boldsymbol{\theta} \in \mathcal{S}^{(j)}$. A continuación se calcula el promedio de las normas euclidianas de estos valores sobre todo el subconjunto:

$$E^{(j)} = \frac{1}{|\mathcal{S}^{(j)}|} \sum_{\boldsymbol{\theta} \in \mathcal{S}^{(j)}} \|\mathbf{F}(\mathbf{g}^{(j)}(\boldsymbol{\theta}); \boldsymbol{\theta})\|_2.$$

Si el error medio $E^{(j)}$ es menor que una tolerancia predefinida ε_{val} , la solución se considera válida. En ese caso, la solución se incorpora a la lista

de resultados definitivos y los puntos del subconjunto $\mathcal{S}^{(j)}$ se retiran del conjunto de datos, ya que han sido explicados por una expresión analítica aceptada.

Si, por el contrario, el error medio supera la tolerancia, la solución se descarta y los puntos correspondientes permanecen en el conjunto de datos, disponibles para ser considerados en iteraciones posteriores del algoritmo. De esta manera, el proceso de validación actúa como un filtro que garantiza que solo se conservan las descripciones que reproducen fielmente el comportamiento del sistema original.

2.3.5. Funciones de pérdida utilizadas

La regresión simbólica emplea una función de pérdida que mide la discrepancia entre las predicciones y los valores observados. En este trabajo se evaluaron tres alternativas.

- **Error cuadrático medio (MSE).** Definido en la sección 2.3.5. Penaliza las desviaciones de forma cuadrática, lo que proporciona una señal suave durante la búsqueda pero lo hace sensible a valores atípicos [5].
- **Pérdida por conteo de coincidencias.** Esta función cuenta directamente la fracción de puntos en los que la predicción difiere del valor real en menos de una tolerancia ε :

$$\mathcal{L}_{\text{match}}(f) = 1 - \frac{1}{N} \sum_{i=1}^N \mathbb{I}(|v_i - f(\mathbf{u}_i)| < \varepsilon).$$

La función indicadora $\mathbb{I}(\cdot)$ vale 1 cuando la condición se cumple y 0 en caso contrario. Por tanto, minimizar $\mathcal{L}_{\text{match}}$ equivale a maximizar el número de puntos que se consideran bien ajustados. Esta métrica refleja de manera directa el objetivo del problema, pero solo distingue dos situaciones —punto ajustado o no ajustado— sin matices intermedios, lo que puede dificultar la convergencia cuando las soluciones candidatas están lejos del objetivo.

- **Pérdida sigmoidal.** Para suavizar la transición que presenta la pérdida por conteo, se introduce una función basada en la sigmoide, también conocida como función logística [1]. Esta función está definida como

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad (9)$$

y produce una curva en forma de “S” que transita suavemente entre 0 y 1. Utilizando esta función, la pérdida sigmoideal se define como

$$\mathcal{L}_{\text{sig}}(f) = 1 - \frac{1}{N} \sum_{i=1}^N \sigma(k(\varepsilon - |v_i - f(\mathbf{u}_i)|)),$$

donde $k > 0$ controla la pendiente de la transición. Cuando $|v_i - f(\mathbf{u}_i)|$ es mucho menor que ε , el argumento de la sigmoide es grande y positivo, por lo que $\sigma(\cdot) \approx 1$ y la contribución a la pérdida es cercana a 0; cuando el error es mucho mayor que ε , el argumento es grande y negativo, $\sigma(\cdot) \approx 0$ y la contribución es cercana a 1. Para errores en una zona intermedia alrededor de ε , la sigmoide toma valores graduales, lo que proporciona al algoritmo una señal más informativa que la simple decisión binaria de la pérdida por conteo [4].

Tras experimentos preliminares, se optó por la pérdida sigmoideal, ya que combina la interpretabilidad de la pérdida por conteo con una señal más rica durante la búsqueda. Los resultados detallados de la comparación entre las tres funciones de pérdida se presentan en el capítulo de experimentación.

2.4. Pseudocódigo del proceso principal

El procedimiento descrito en las secciones anteriores se integra en un algoritmo que toma como punto de partida la definición del sistema de ecuaciones y devuelve las expresiones analíticas para cada variable en cada una de las soluciones encontradas. A continuación se presenta el pseudocódigo global.

Pseudocódigo 1:

- Entrada:** sistema $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}$, variables $\mathbf{x}$,
parámetros $\boldsymbol{\theta}$, intervalos para parámetros y para
puntos iniciales, tolerancias, hiperparámetros de PySR
- Salida:** lista de soluciones, cada una con una expresión
simbólica por variable
1. Definir cuadrícula de parámetros $\mathcal{G}$ mediante producto cartesiano (sección 2.2).
 2. Para cada $\boldsymbol{\theta} \in \mathcal{G}$:
 Resolver numéricamente con múltiples puntos iniciales (sección 2.2).
 Filtrar soluciones duplicadas usando distancia euclidiana.
 3. Combinar todos los pares $(\boldsymbol{\theta}, \mathbf{x})$ en un solo conjunto de datos (sección 2.3.1).

4. Tomar x_1 como guía para separar las soluciones (sección 2.3.1).
5. Aplicar búsqueda iterativa sobre x_1 (sección 2.3.2) para obtener una lista de expresiones $g_1^{(1)}, g_1^{(2)}, \dots$ y los subconjuntos de parámetros que cada una cubre.
6. Para cada expresión $g_1^{(j)}$ y su subconjunto de parámetros $\mathcal{S}^{(j)}$:
 - Para cada coordenada restante x_k con $k \neq 1$:
 - Entrenar regresión simbólica sobre $\mathcal{S}^{(j)}$ (sección 2.3.3).
 - Construir la función completa $\mathbf{g}^{(j)}(\boldsymbol{\theta})$.
 - Validar calculando el error medio (sección 2.3.4).
 - Si la validación es satisfactoria, añadir la solución a la salida y retirar los puntos de $\mathcal{S}^{(j)}$ del conjunto de datos.
7. Repetir los pasos 5–6 hasta que el número de puntos restantes sea menor que un umbral mínimo.

Los hiperparámetros mencionados (tolerancias, número de intentos, parámetros de PySR) se detallan en el capítulo de experimentación, donde se justifican los valores seleccionados.

2.5. Implementación del sistema

El algoritmo descrito en las secciones anteriores se materializa en un sistema compuesto por varios módulos independientes, cada uno responsable de una etapa del proceso. A continuación se describen estos módulos, se detallan los recursos que requieren y se resume el formato de los resultados que producen.

2.5.1. Descripción de los módulos

La implementación sigue el orden del pipeline presentado en el pseudocódigo de la sección 2.4. Los módulos principales son:

- `config.py`: centraliza todos los parámetros globales del experimento, como los intervalos de los parámetros, el número de puntos de la cuadrícula, las opciones del solucionador numérico (tolerancias, número de intentos) y los hiperparámetros del modelo de regresión simbólica

(tamaño de la población, número de generaciones, coeficiente de parsimonia, etc.).

- `equation_parser.py`: convierte las ecuaciones del sistema, escritas como cadenas de texto, en expresiones simbólicas de `SymPy` [?] que pueden ser evaluadas numéricamente. Para sistemas de varias ecuaciones, este módulo procesa la lista completa y devuelve tanto las expresiones como un diccionario con todos los símbolos involucrados.
- `cuadrícula_generator.py`: construye el producto cartesiano de los conjuntos de valores de los parámetros (sección 2.2), generando la cuadrícula $\mathcal{G}$ que se utiliza como entrada en la etapa de resolución numérica.
- `solver.py`: para cada tupla de la cuadrícula, resuelve numéricamente el sistema con `SciPy` [9] empleando múltiples puntos iniciales aleatorios. Aplica el filtro de duplicados basado en la distancia euclidiana (sección 1.1) y devuelve la lista de soluciones únicas encontradas para cada valor de los parámetros.
- `root_grouping.py`: reorganiza los datos generados por el solucionador para que puedan ser utilizados por los módulos de regresión simbólica. Produce tanto el conjunto combinado de pares $(\theta, \mathbf{x})$ como, opcionalmente, la separación por soluciones de acuerdo con la estrategia descrita en la sección 2.3.1.
- `symbolic_regression.py`: contiene la lógica de entrenamiento de `PySR` [4] y el algoritmo iterativo que, partiendo del conjunto de datos completo, descubre una a una las expresiones para x_1 (Sección 2.3.2). Tras cada entrenamiento, se examinan todas las ecuaciones generadas por el modelo y se selecciona aquella que cubre la mayor cantidad de puntos, descartando el resto antes de la siguiente iteración.
- `regression_adapter.py`: implementa la estrategia multidimensional: extiende las expresiones obtenidas para x_1 a las coordenadas restantes (sección 2.3.3) y valida cada solución completa calculando el error medio sobre el sistema original (sección 2.3.4).
- `main.py`: integra los módulos anteriores y ejecuta el pipeline completo, desde la lectura de la configuración hasta la escritura de los resultados.

Además de los módulos desarrollados, el sistema depende de las bibliotecas externas `SymPy` para la manipulación simbólica, `SciPy` para la resolución numérica y `PySR` para la regresión simbólica.

2.5.2. Entradas, parámetros y salidas del sistema

Para cerrar la descripción del método, se resumen a continuación los datos que requiere el sistema y los resultados que produce.

Entradas del sistema:

- Lista de ecuaciones que definen el sistema $\mathbf{F}(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{0}$.
- Nombres de las variables $\mathbf{x}$ y de los parámetros $\boldsymbol{\theta}$.
- Intervalos $[\theta_i^{\min}, \theta_i^{\max}]$ para cada parámetro, y número de puntos de la cuadrícula.

Parámetros principales:

- Número de puntos iniciales aleatorios y tolerancias para la resolución numérica ($\varepsilon_{\text{res}}, \tau$).
- Función de pérdida seleccionada y sus parámetros (ε, k) (sección 2.3.5).
- Hiperparámetros de PySR: tamaño de población, número de generaciones, coeficiente de parsimonia, tamaño máximo de árbol, etc. (sección 1.2).

Salidas del sistema:

- Lista de soluciones encontradas. Cada solución consiste en un conjunto de expresiones simbólicas $\{g_1(\boldsymbol{\theta}), \dots, g_n(\boldsymbol{\theta})\}$ que describen las variables en función de los parámetros.
- Para cada solución se indica el subconjunto de parámetros sobre el cual fue validada y el número de puntos que cubre.

Con esta arquitectura, el sistema es capaz de procesar un sistema de ecuaciones no lineales paramétrico, generar los datos de entrenamiento, aplicar la estrategia de regresión simbólica descrita y producir expresiones analíticas para las soluciones. En el siguiente capítulo se presentan los experimentos realizados para evaluar el desempeño del método propuesto y la influencia de los distintos parámetros de configuración.

3. Experimentación y resultados

4. Conclusiones

Referencias

- [1] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [2] Charles G. Broyden. A class of methods for solving nonlinear simultaneous equations. *Mathematics of Computation*, 19(92):577–593, 1965.
- [3] Richard L. Burden and J. Douglas Faires. *Numerical Analysis*. Brooks/Cole, 9th edition, 2010.
- [4] Miles Cranmer. PySR: Fast & parallelized symbolic regression in Python/Julia. *Journal of Open Source Software*, 8(89):5308, 2023.
- [5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2nd edition, 2009.
- [6] John R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, USA, 1992.
- [7] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [8] Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, 2009.
- [9] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.